



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

Report di Progetto (v1.4):

PROMUSIC WEB APP

Programmazione per il Web – A.A. 2024/25
Informatica e Tecnologie per la Produzione del Software
Data del report: 6 Settembre 2025

Realizzato da:



HiddenBit
Gruppo A

Manzoni Matteo 775527 m.manzoni3@studenti.uniba.it
Marvulli Antonio 787058 a.marvulli18@studenti.uniba.it

SOMMARIO

<u>1.</u>	<u>DESCRIZIONE INFORMALE DEL CONTESTO E DEGLI OBIETTIVI.....</u>	<u>4</u>
<u>2.</u>	<u>INDIVIDUAZIONE DEGLI STAKEHOLDER E DEI GOAL.....</u>	<u>5</u>
2.1	Individuazione goal amministratore.....	5
2.2	Diagramma di contesto.....	6
2.3	Scambio di valore.....	6
<u>3.</u>	<u>STATO DELL'ARTE, ANALISI DELLA CONCORRENZA E ANALISI "MAKE OR BUY".....</u>	<u>7</u>
3.1	Stato dell'arte.....	7
3.2	Analisi della concorrenza.....	7
3.3	Analisi "Make or Buy".....	10
<u>4.</u>	<u>MODELLO INFORMATIVO.....</u>	<u>12</u>
4.1	Diagramma EER.....	12
4.2	Modello relazionale.....	13
4.3	Stima dimensionale del DB.....	13
4.3.1	Stima dimensionale per entità.....	13
4.3.2	Tabella popolamento iniziale.....	14
4.3.3	Crescita annuale.....	14
4.4	Modello di visibilità per amministratore.....	15
<u>5.</u>	<u>MODELLO DI NAVIGAZIONE E PRESENTAZIONE.....</u>	<u>16</u>
5.1	Modello di navigazione.....	16
5.2	Modello di presentazione.....	16
<u>6.</u>	<u>MODELLO ARCHITETTURALE.....</u>	<u>17</u>
6.1	Hardware.....	17
6.2	Software.....	17
6.2.1	Application server.....	17
6.2.2	Librerie principali.....	18
6.2.3	Database Management System.....	18
6.2.4	Browser supportati.....	18
6.2.5	Sistemi operativi supportati.....	19
<u>7.</u>	<u>ASPETTI IMPLEMENTATIVI.....</u>	<u>20</u>
7.1	Client.....	20

7.1.1	client/src/.....	20
7.1.2	client/src/components/.....	20
7.1.3	client/src/scenes/.....	21
7.1.4	client/src/state/.....	22
7.2	Server.....	23
<u>8.</u>	<u>TEST.....</u>	<u>24</u>
<u>9.</u>	<u>CONCLUSIONI.....</u>	<u>25</u>
<u>10.</u>	<u>MATRICE RACI.....</u>	<u>26</u>

1. DESCRIZIONE INFORMALE DEL CONTESTO E DEGLI OBIETTIVI

La nostra applicazione mira ad offrire una piattaforma di vendita al dettaglio moderna ed innovativa grazie ad una attenta analisi di mercato e l'utilizzo di intelligenza artificiale, che possano essere in grado di soddisfare qualsiasi esigenza del cliente. Il committente è, nello specifico, un affermato negozio di musica nella città di Bari.

Puntiamo quindi a realizzare un sito web sul quale il commerciante potrà inserire i propri prodotti e le relative proposte di vendita. In particolare, questi faranno parte di un ampio catalogo comprendente strumenti musicali, attrezzatura per concerti, e dispositivi per registrazione e riproduzione audio.

Oltre alla parte dedicata agli utenti, sarà realizzato un pannello amministratore intuitivo e completo. Il suo obiettivo sarà quello di permettere al commerciante di gestire in maniera centralizzata l'intero catalogo, monitorare l'andamento delle vendite e ottenere una panoramica dettagliata sulle preferenze degli utenti.

Il nostro obiettivo è quello di offrire un'interfaccia moderna e accattivante, senza trascurarne l'aspetto funzionale: il punto di forza dell'applicazione sarà proprio l'inedito utilizzo di un chatbot intelligente, in grado di servire l'utente interessato all'acquisto proprio come se questi stesse interagendo con un vero commesso in carne ed ossa.

Sarà possibile chiedere consigli in merito ai prodotti, in modo da capire quale sia il più adatto alle proprie esigenze; ricercarne uno specifico in base alle proprie necessità e desideri; infine, si potrà ricevere assistenza sugli articoli già acquistati.

Crediamo fortemente che questo possa rappresentare un grande vantaggio competitivo rispetto gli altri siti di e-commerce nello stesso settore, che ancora non dispongono di tali funzionalità. Molte persone potrebbero non avere il tempo di recarsi presso la struttura fisica dell'attività, altre potrebbero avere impedimenti motori o essere semplicemente troppo distanti.

La tecnologia sta evolvendo ed il mondo con essa, perché non sfruttarla?

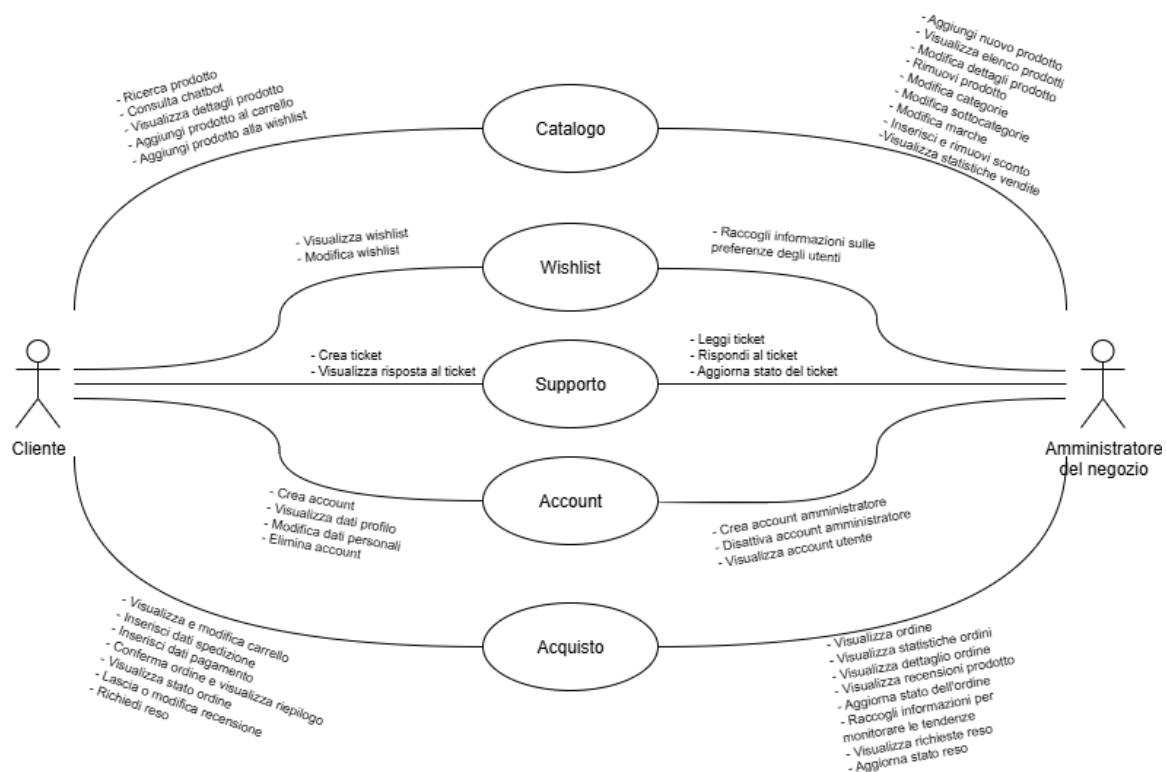
2. INDIVIDUAZIONE DEGLI STAKEHOLDER E DEI GOAL

2.1 Individuazione goal amministratore

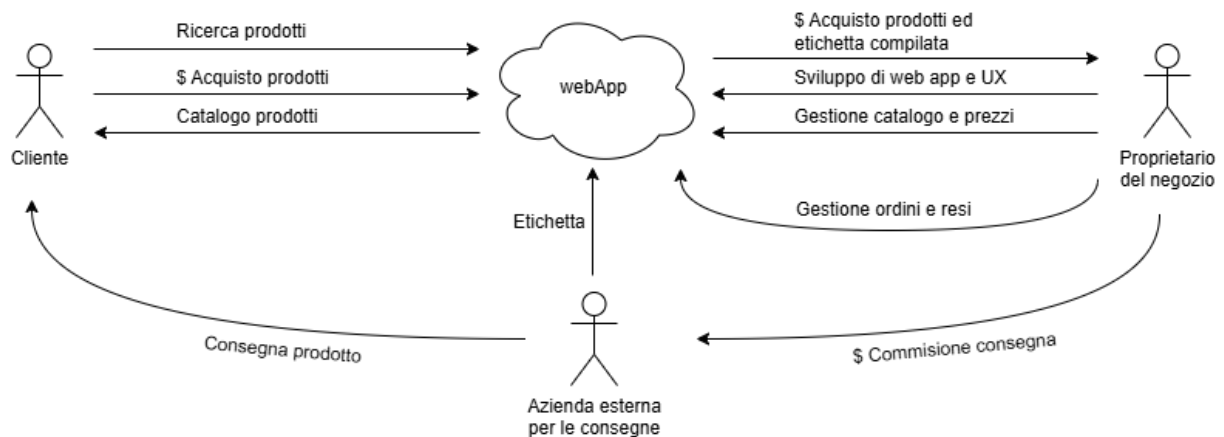
Amministratore del negozio: gestore della piattaforma web e responsabile delle funzionalità con privilegi su catalogo, ordini e supporto clienti.

STAKEHOLDER	GOAL	SUBGOAL	C	R	U	D
AMMINISTRATORE DEL NEGOZIO	Gestione catalogo	Aggiungi nuova istanza nel catalogo	✓			
		Visualizza catalogo		✓		
		Modifica dettagli istanza del catalogo			✓	
		Rimuovi istanza del catalogo				✓
	Gestione ordini	Visualizza ordini ricevuti		✓		
		Visualizza dettaglio ordine		✓		
		Aggiorna stato ordine			✓	
		Gestione richieste di reso			✓	
	Supporto tecnico	Lettura ticket		✓		
		Risoluzione ticket			✓	
	Statistiche e monitoraggio	Visualizza statistiche vendite		✓		
		Visualizza recensioni utenti		✓		
		Visualizza statistiche wishlist		✓		
		Visualizza statistiche ordini		✓		

2.2 Diagramma di contesto



2.3 Scambio di valore



3. STATO DELL'ARTE, ANALISI DELLA CONCORRENZA E ANALISI "MAKE OR BUY"

3.1 Stato dell'arte

Negli ultimi anni i software gestionali hanno conosciuto un'evoluzione profonda, trasformandosi da semplici strumenti di supporto amministrativo a piattaforme fondamentali ad ogni tipo di impresa. Inizialmente, questi sistemi erano progettati per supportare l'azienda in attività semplici ed isolate. Oggi, invece, offrono un supporto completo su numerosi processi aziendali permettendo di avere un controllo centralizzato sull'intera organizzazione.

Al centro di questi sistemi si trova un database unificato, che raccoglie e rende disponibili tutte le informazioni aziendali in tempo reale. Questo consente di ridurre errori, offrendo all'azienda possibilità di prendere decisioni più rapide e precise.

Un aspetto distintivo dei gestionali moderni è la loro modularità. Ogni impresa può attivare solo i moduli di interesse in base alle proprie esigenze. Questa caratteristica rende i software gestionali scalabili e adattabili sia alle piccole e medie imprese quanto alle grandi aziende.

Inoltre, i gestionali odierni, possono integrarsi con altri sistemi esterni come piattaforme e-commerce, software di logistica e strumenti di analisi avanzata. Questa capacità permette una gestione dei flussi aziendali efficiente.

Molti software gestionali moderni includono strumenti avanzati di intelligenza artificiale e machine learning, che automatizzano compiti ripetitivi e analizzano dati aziendali fornendo previsioni sulle vendite. In alcuni casi è anche utilizzata per supportare l'utente interno con chatbot e assistenti virtuali per lo svolgimento di attività quotidiane.

La sicurezza dei dati rappresenta un ulteriore punto di forza per i gestionali moderni, poiché trattano informazioni estremamente sensibili, quali dati dei dipendenti, informazioni sui clienti o ordini. Per questo motivo, vengono utilizzati protocolli di crittografia e sistemi di autenticazione avanzata per garantire la protezione delle informazioni.

Infine, i gestionali non sono più semplici strumenti di supporto, ma si presentano come piattaforme dinamiche e flessibili, capaci di adattarsi ai cambiamenti di mercato o alle esigenze dell'impresa. Essi consentono di personalizzare flussi di lavoro e avere un punto di accesso per il monitoraggio della realtà aziendale. Questo rende un sistema gestionale un vero e proprio strumento strategico, capace di supportare la crescita dell'impresa.

3.2 Analisi della concorrenza

Qui di seguito vengono riportati gli e-commerce di maggior rilievo nel contesto del mercato attuale, analizzando i competitors sia a livello locale (piccoli negozi) che internazionale (grandi aziende e marketplace).

Thomann: con sede in Germania, è il punto di riferimento europeo per la vendita online di strumenti e attrezzature musicali, vanta un catalogo di oltre 113.000 prodotti; l'interfaccia è altamente interattiva e dal design chiaro e moderno, e presenta una homepage ricca ma ordinata: grandi categorie, offerte in evidenza, sezioni per novità o strumenti consigliati. L'utente può cercare un prodotto specifico, filtrare per tipo, marca, prezzo o recensioni, e confrontare più articoli tra loro. Include un'area personale molto completa dove controllare gli ordini, scaricare fatture, gestire resi e richiedere assistenza. È anche possibile creare liste

dei desideri, ricevere suggerimenti personalizzati e salvare le preferenze di acquisto. In più, la piattaforma offre numerosi contenuti di supporto, come video dimostrativi, recensioni tecniche, guide all'acquisto e FAQ dettagliate. Supporta un'interfaccia in più lingue, prezzi in diverse valute, e spedizioni globali, con calcolo automatico di dazi, IVA e dogana, nonché supporto clienti localizzato. Non ha una sezione dedicata all'usato.

Gear4music: piattaforma inglese progettata per chi vuole fare un acquisto rapido, senza perdersi in troppi dettagli. La homepage mette subito in risalto offerte, prodotti consigliati, e categorie popolari. L'esperienza di navigazione è intuitiva e leggera, anche su dispositivi mobili. Le descrizioni sono chiare, con tabelle di specifiche essenziali e poche distrazioni. Non ci sono molte funzionalità "avanzate", ma tutto è estremamente funzionale. Supporta un'interfaccia in più lingue.

StrumentiMusicali.net: e-commerce italiano e focalizzato sul mercato nazionale, ha un catalogo meno vasto rispetto ad altre piattaforme più affermate come quella di Thomann, un'interfaccia meno moderna e presenta meno funzionalità avanzate. Tuttavia offre una navigazione semplice e intuitiva, proponendo un valido assortimento di prodotti (comprensivo di marchi italiani difficilmente reperibili altrove), e delle spedizioni rapide e poco costose sul territorio italiano. Non offre un'interfaccia multilingua.

AcusticaBari: italiano e con negozio fisico situato nella città di Bari, si rivolge principalmente al mercato locale e regionale. Ha un sito semplice e poco interattivo, con una struttura basilare che presenta i prodotti disponibili attraverso un'interfaccia limitata. Il catalogo è più scarso rispetto ai precedenti ed è organizzato in maniera approssimativa, i filtri per la ricerca sono quelli essenziali. Il sito fornisce informazioni sui prodotti, ma non sembra essere orientato all'e-commerce. Essendo un negozio locale, offre un servizio più personale e diretto, con la possibilità di provare i prodotti in loco e ricevere consigli immediati. È possibile selezionare la lingua inglese.

Amazon: piattaforma di successo globale che non ha bisogno di presentazioni, vende praticamente di tutto, strumenti musicali inclusi, ma non è specializzata in quel settore. Si possono trovare quindi tantissimi prodotti, spesso a prezzi competitivi, e approfittare di servizi efficienti come Prime per spedizioni rapide e resi semplificati; dal punto di vista della web app, tuttavia, l'esperienza su Amazon è molto "anonima": la descrizione dei prodotti può essere scarsa, le schede tecniche a volte incomplete, e l'assistenza clienti è generica, non specializzata. Inoltre non sono presenti filtri di ricerca specifici per strumenti e attrezzatura musicale, e altre funzionalità avanzate proprie delle piattaforme di e-commerce dedicate a questo scopo.

Reverb: marketplace peer-to-peer per vendita di strumenti musicali usati con sede negli USA. Ha un'interfaccia moderna e minimalista, con un homepage che mette in risalto i prodotti attraverso l'utilizzo di grandi immagini disposte in un layout a griglia. Le funzionalità principali per l'utente registrato includono la creazione di un proprio profilo venditore, la pubblicazione di annunci con descrizioni, foto e prezzo personalizzato e la possibilità di fare offerte e trattare direttamente con chi vende. Per chi acquista, la ricerca è facilitata da filtri molto flessibili (condizione dello strumento, paese, prezzo, categoria) ma non troppo specifici. Il sito include recensioni e feedback sui venditori. Supporta un'interfaccia multilingua.

3.3 Analisi “Make or Buy”

Nel contesto di sviluppo della web app non bisogna sottovalutare la sezione amministrativa. A tal proposito è stata condotta un'attenta analisi “Make or Buy” volta a confrontare entrambe le possibili soluzioni al fine di individuare l'approccio più adatto alle esigenze specifiche del progetto.

La sezione amministrativa riveste un ruolo cruciale, in quanto coinvolge aspetti come la gestione del catalogo, dei prezzi, degli ordini e dei resi, oltre all'analisi statistica dell'andamento dell'e-commerce. Si tratta quindi di un'area che richiede robustezza, flessibilità, affidabilità e possibilità di personalizzazione.

Nella fase di valutazione delle opzioni “Buy”, sono stati esaminati quattro CMS e-commerce tra i più diffusi: Shopify, WooCommerce, Wix e Adobe Commerce con i relativi pro e contro.

SOLUZIONE	PRO	CONTRO
SHOPIFY	• UI moderna e intuitiva • Personalizzazione semplificata	• Personalizzazione limitata • Costi periodici • Funzionalità avanzate a pagamento • Richiede manutenzione periodica
WooCommerce (WordPress Plugin)	• Gratuito • Ampia personalizzazione	• Prestazioni legate all'hosting di WordPress • Funzionalità avanzate richiedono plugin a pagamento • Richiede manutenzione periodica
Wix eCommerce	• Facilità d'uso • Manutenzione minima	• Limitata personalizzazione • Non adatto a WebApp con funzionalità avanzate
Adobe Commerce	• Ampia personalizzazione • Adatto per funzionalità avanzate	• Non adatto a neofiti senza conoscenze informatiche • Alti costi di licenza

L'analisi mette in evidenza i limiti esistenti in termini di personalizzazione e rapporto costi/benefici che rendono queste piattaforme non idonee allo sviluppo della sezione amministrativa della WebApp.

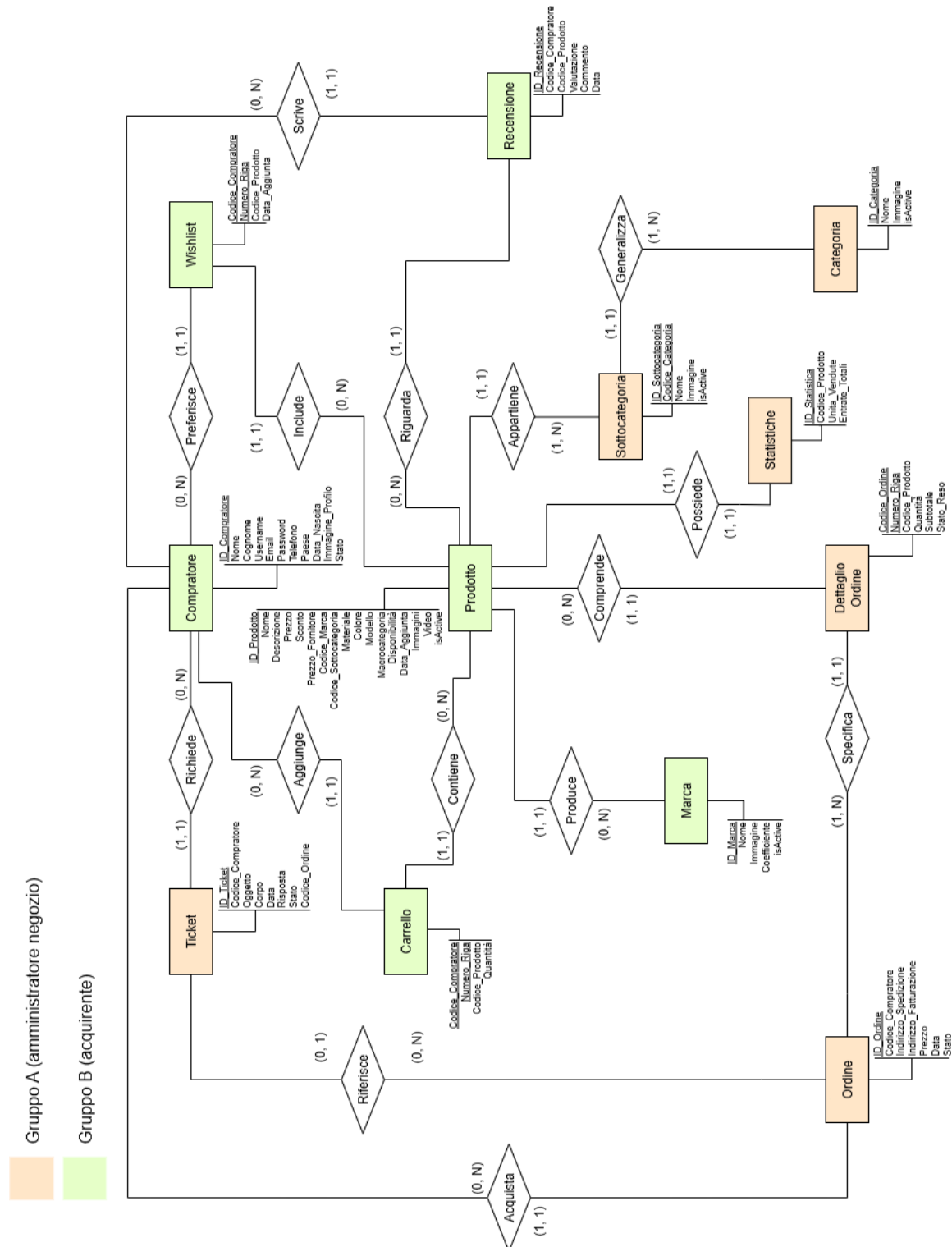
Alla luce di queste considerazioni, è stato deciso di adottare un approccio "Make", ovvero lo sviluppo dell'applicazione partendo da "zero".

Quest'ultimo, infatti, consente di progettare un sistema pienamente personalizzato e adatto alle esigenze del committente con un controllo diretto sull'architettura del sistema rendendo possibile aggiornamenti futuri, senza scendere a compromessi nella realizzazione delle componenti.

Tale approccio, nonostante comporti inizialmente un investimento maggiore in termini di tempo e risorse, risulta strategicamente più vantaggioso sul lungo periodo, garantendo un prodotto finale pienamente aderente alle necessità del progetto.

4. MODELLO INFORMATIVO

4.1 Diagramma EER



4.2 Modello relazionale

Gruppo A (amministratore negozio)

Gruppo B (acquirente)



4.3 Stima dimensionale del DB

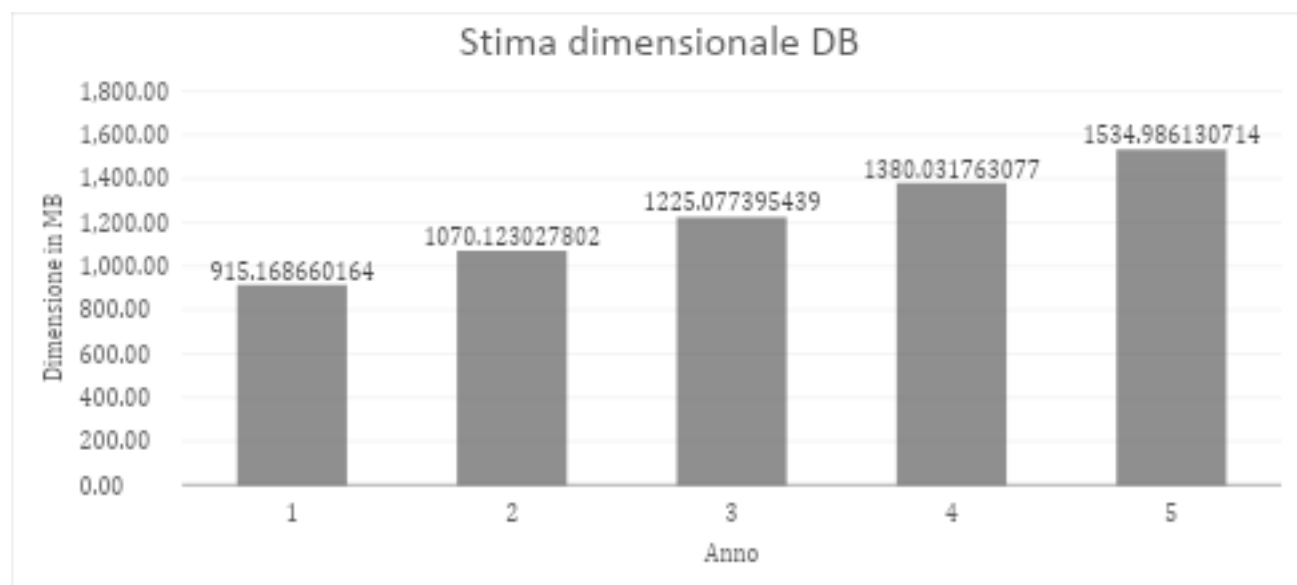
4.3.1 Stima dimensionale per entità

[Stima dimensionale per entità.](#)

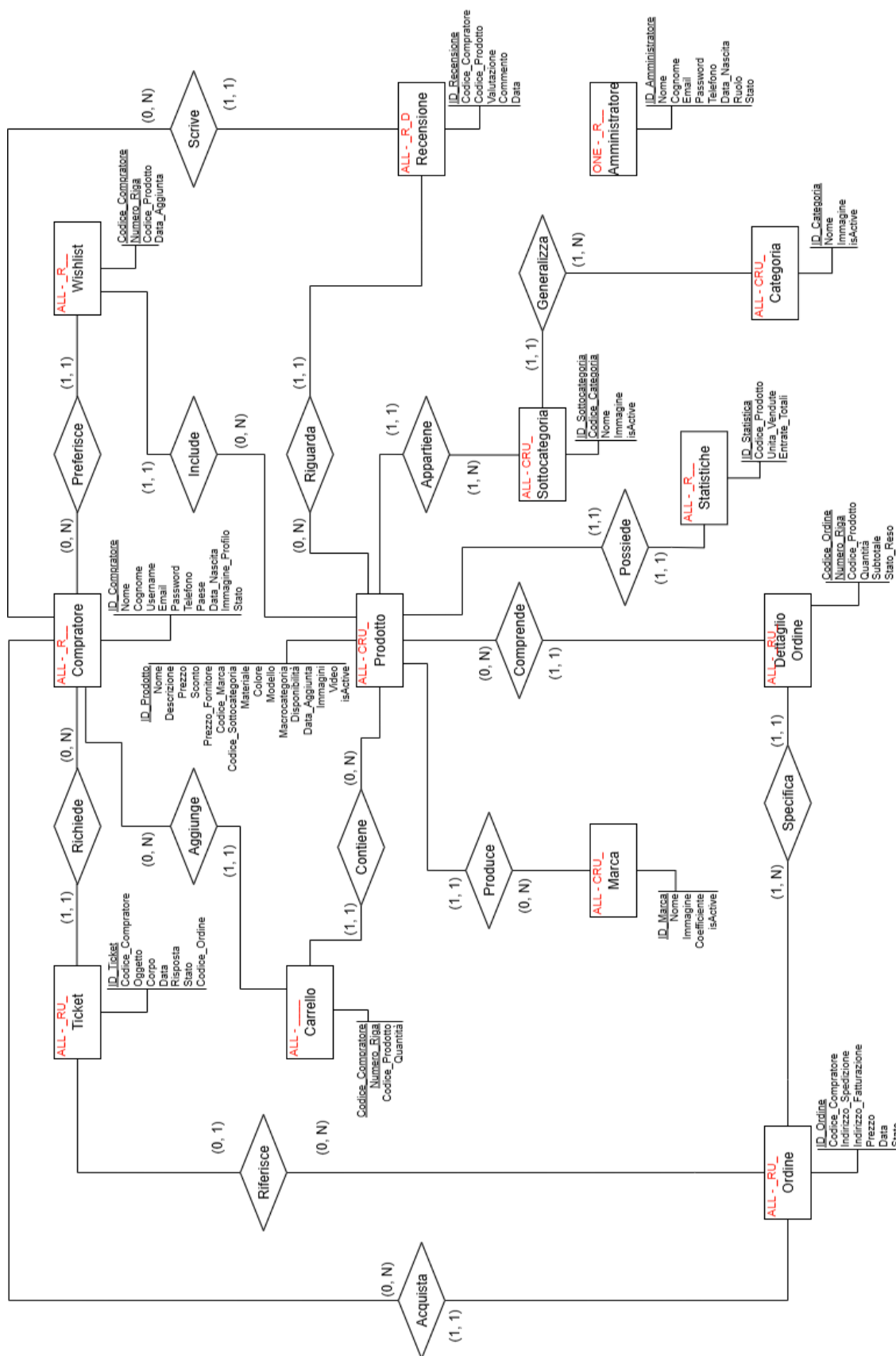
4.3.2 Tabella popolamento iniziale

TIPO ENTITÀ	DIMENSIONE RECORD (BYTE)	POPOLAMENTO INIZIALE	TASSO DI CRESCITA ANNUALE
Compratore	200000	1000	+500
Prodotto	600000	800	+100
Ordine	91	1200	+1000
Dettaglio Ordine	34	2400	+2000
Carrello	24	300	+200
Wishlist	23	2000	+1000
Ticket	454	100	+150
Recensione	132	400	+200
Categoria	200000	12	+1
Sottocategoria	200000	226	+5
Amministratore	131	2	+1
Marca	200000	346	+5
Statistiche	22	800	+100

4.3.3 Crescita annuale

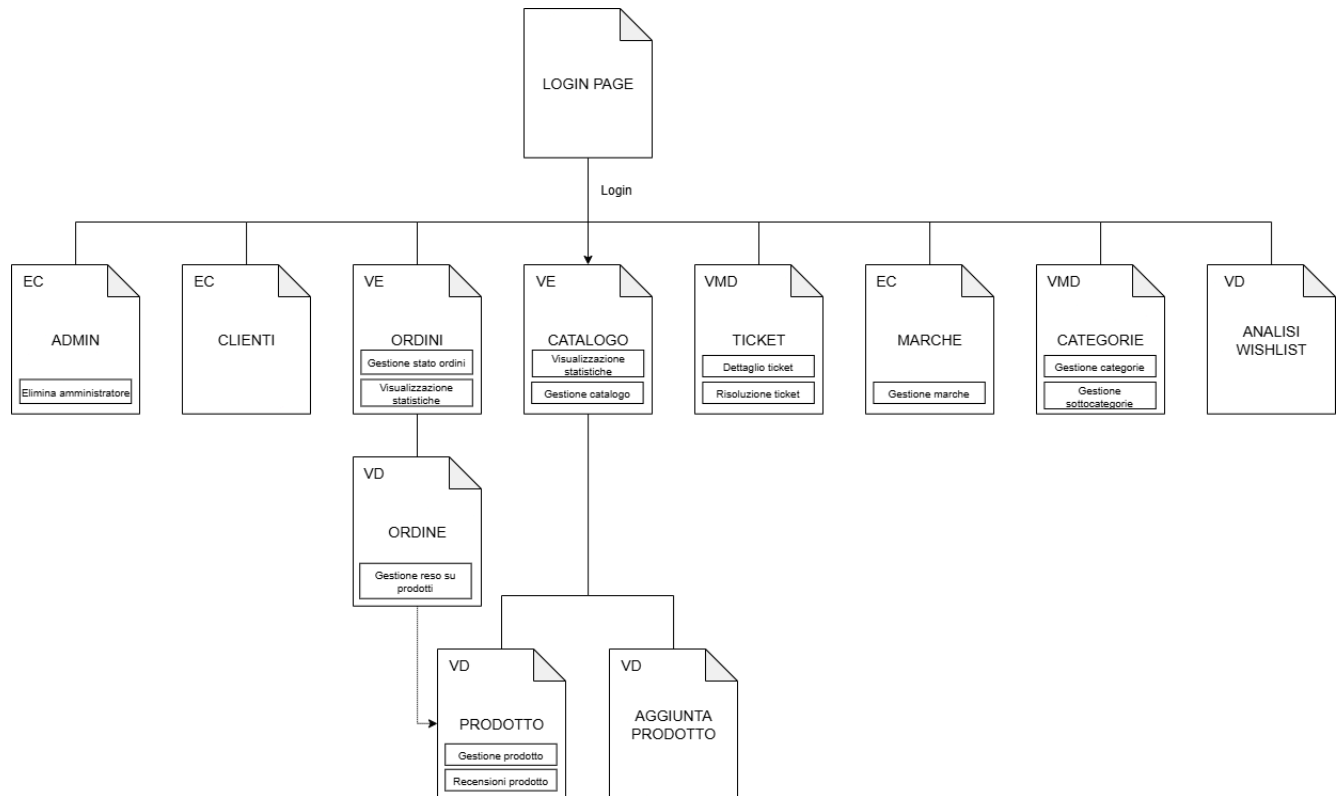


4.4 Modello di visibilità per amministratore



5. MODELLO DI NAVIGAZIONE E PRESENTAZIONE

5.1 Modello di navigazione

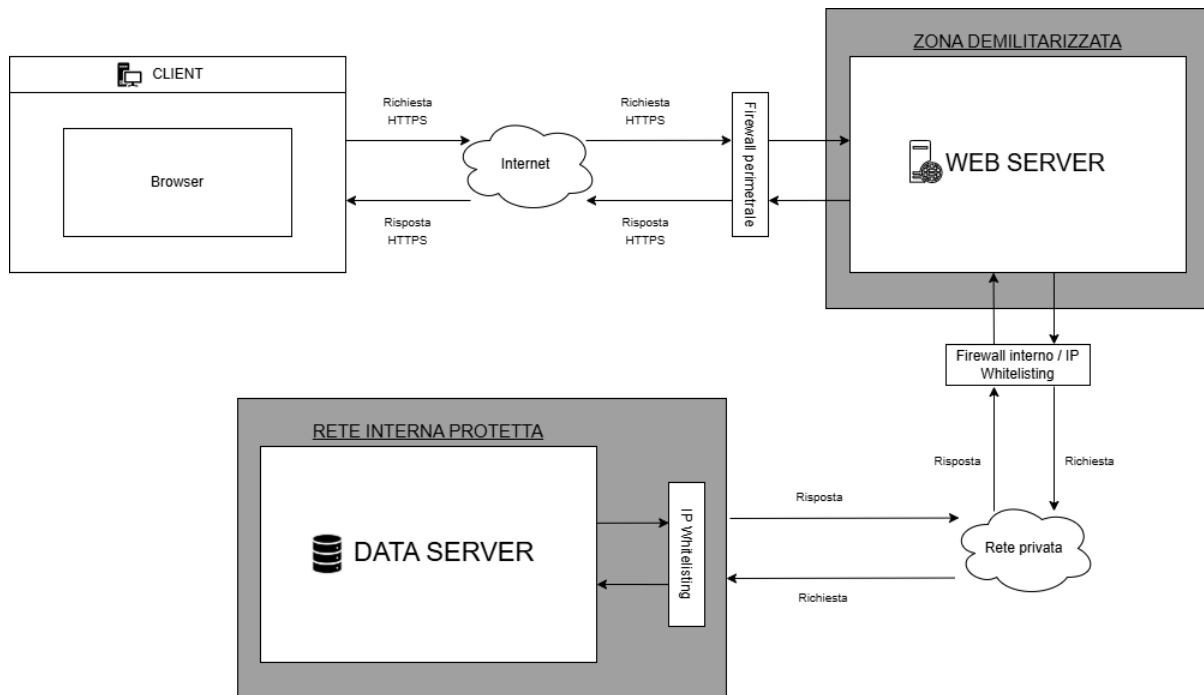


5.2 Modello di presentazione

[Modello di presentazione amministratore.](#)

6. MODELLO ARCHITETTURALE

6.1 Hardware



6.2 Software

Il sistema è stato realizzato secondo il modello stack MERN (MongoDB, Express.js, React, Node.js). Questo approccio è stato scelto per una migliore modularità, scalabilità e manutenibilità del codice.

6.2.1 Application server

È la parte principale del back-end, che si occupa di ricevere richieste dal client, interagire con il database e restituire le risposte.

E' formato da:

- **Node.js (v. 22.17.0)** □ Ambiente runtime di JavaScript che permette l'esecuzione di codice JS, lato server, al di fuori del browser.
- **Express.js (v. 5.1.0)** □ Framework per Node.js, utilizzato per la gestione delle richieste HTTP, definizione di rotte e integrazione di middleware.

6.2.2 Librerie principali

Front-end

- **React (v. 19.1.0)** □ Libreria JavaScript per la creazione di interfacce utente reattive e componentizzate.
- **React Redux (v. 9.2.0)** □ Libreria che integra Redux con React, permettendo la gestione centralizzata dello stato dell'applicazione e facilitando la condivisione tra i vari componenti.
- **React Router DOM (v. 7.6.3)** □ Libreria per la gestione del routing lato client, consentendo la navigazione tra pagine senza ricaricare l'intera app.
- **Material UI (v. 7.2.0)** □ Libreria per componenti grafici moderni e responsive.
- **Nivo (v. 0.99.0)** □ Libreria per la creazione di grafici interattivi.
- **Formik (v. 2.4.6)** □ Libreria per la gestione dei form e validazione dei dati.
- **Yup (v. 1.7.0)** □ Libreria per la validazione di dati tramite form.

Back-end

- **Bcrypt (v. 6.0.0)** □ Libreria per l'hashing sicuro delle password.
- **JSON Web Token (v. 9.0.2)** □ Libreria per la gestione dell'autenticazione.
- **Multer (v. 2.0.2)** □ Middleware per la gestione dell'upload di file.
- **Cloudinary (v. 2.7.0)** □ Libreria per l'archiviazione in cloud ed elaborazione di immagini.
- **Nodemailer (v. 7.0.6)** □ Libreria che permette l'invio di email direttamente dal server in modo sicuro e automatizzato.
- **Mongoose (v. 8.17.1)** □ Object Data Modelling (ODM) che semplifica la definizione di schemi, la validazione dei dati e l'interazione con il DBMS MongoDB.
- **Helmet (v. 8.1.0)** □ Middleware che migliora la sicurezza delle applicazioni web impostando automaticamente intestazioni HTTP protettive.
- **Morgan (v. 1.10.1)** □ Middleware che effettua il logging delle richieste HTTP.
- **Cookie-parser (v. 1.4.7)** □ Middleware che analizza e gestisce i cookie presenti nelle richieste HTTP, facilitando l'accesso a dati memorizzati.
- **CORS (v. 2.8.5)** □ Middleware che gestisce le policy di accesso cross-origin, permettendo al front-end di comunicare con il back-end in sicurezza da domini diversi.

6.2.3 Database Management System

- **MongoDB (v. 8.0.13)** □ Database utilizzato per l'archiviazione dei dati applicativi.

L'accesso al database è gestito tramite mongoose, che fornisce schemi e modelli per il suo corretto utilizzo.

6.2.4 Browser supportati

L'applicazione è supportata dai seguenti browser:

- **Google Chrome**
- **Mozilla Firefox**
- **Microsoft Edge**
- **Brave**

6.2.5 Sistemi operativi supportati

L'applicazione sarà supportata dai seguenti sistemi operativi:

- **Android**
- **iOS**
- **Windows**
- **macOS**

7. ASPETTI IMPLEMENTATIVI

7.1 Client

La cartella client si divide in 4 percorsi principali:

- [client/src/](#)
- [client/src/components/](#)
- [client/src/scenes/](#)
- [client/src/state/](#)

7.1.1 client/src/

N.B. Il colore rosso è associato a **Manzoni Matteo** e il colore verde a **Marvulli Antonio** per la divisione delle responsabilità, in caso di responsabilità condivisa il colore sarà neutrale.

- **App.jsx** □ Componente principale dell'applicazione React, si occupa di instradare le pagine attraverso React Router, della persistenza dell'access token e della condivisione del tema principale tra le varie pagine.
- **main.jsx** □ Componente che rappresenta il punto di ingresso dell'applicazione, si occupa di renderizzare il componente App.jsx e di configurare il Redux Store.
- **theme.jsx** □ Modulo che definisce la palette di colori per entrambi i temi e generare un tema MUI coerente con la modalità scelta.

7.1.2 client/src/components/

N.B. Il colore rosso è associato a **Manzoni Matteo** e il colore verde a **Marvulli Antonio** per la divisione delle responsabilità, in caso di responsabilità condivisa il colore sarà neutrale.

Questo percorso contiene tutti i componenti riutilizzabili da più pagine.

- **AggiungiDialog.jsx** □ Componente che fornisce un pop-up per l'aggiunta di una marca, categoria o sottocategoria, e si occupa della validazione dei dati inviati al back-end.
- **CampoNumerico.jsx** □ Componente che fornisce un campo numerico arricchito con bottoni per l'incremento o il decremento del valore.
- **ConfermaDialog.jsx** □ Componente che fornisce un pop-up di conferma per diverse azioni come la disattivazione o l'attivazione di un prodotto, categoria, sottocategoria o marca, o l'eliminazione di un admin.
- **FlexBetween.jsx** □ Componente funzionale che giustifica secondo definite regole il contenuto al suo interno.
- **GlobalSnackbar.jsx** □ Componente che gestisce e mostra le notifiche all'utente, utilizzato in App.jsx per renderlo default di ogni componente.
- **Header.jsx** □ Componente funzionale per l'inserimento di un header indicativo nelle varie pagine.
- **Layout.jsx** □ Componente che definisce la struttura generale dell'applicazione includendo navbar, sidebar e la pagina corrente.
- **CampoImmagini.jsx** □ Componente che fornisce un campo per l'inserimento di immagini dal pc e opzionalmente un pulsante di invio form.

- **MasterAdminPrivateRoute.jsx** □ Componente funzionale che permette di raggiungere una determinata route solo se il ruolo dell'amministratore equivale a quello del proprietario del negozio.
- **ModificaDialog.jsx** □ Componente che fornisce un form per la modifica di una marca, categoria o sottocategoria, e si occupa della validazione dei dati inviati al back-end.
- **Navbar.jsx** □ Componente che fornisce una barra situata al di sopra delle pagine per fornire funzioni di logout, ricerca catalogo e cambio tema.
- **PrivateRoute.jsx** □ Componente funzionale che permette di raggiungere tutte le route dell'applicazione solo se è stato effettuato il login.
- **Sidebar.jsx** □ Componente che fornisce una barra situata lateralmente che fornisce funzioni di navigazione tra le varie pagine.
- **TopProdottiChart.jsx** □ Componente che fornisce un istogramma per la visualizzazione di varie statistiche come i prodotti più venduti, quelli che hanno generato più profitto e quelli che sono più desiderati.

7.1.3 client/src/scenes/

N.B. Il colore rosso è associato a **Manzoni Matteo** e il colore verde a **Marvulli Antonio** per la divisione delle responsabilità, in caso di responsabilità condivisa il colore sarà neutrale.

Questo percorso contiene i componenti che rappresentano le pagine del sito:

- **AggiungiProdotto.jsx (Aggiungi Prodotto/)** □ Componente che rappresenta la pagina dedicata all'inserimento di un nuovo prodotto.
- **AggiungiAmministratoreDialog.jsx (AggiungiAmministratore/)** □ Componente che rappresenta un pop-up per aggiungere un nuovo amministratore, validando automaticamente i dati inviati al back-end.
- **ActivationPage.jsx (AggiungiAmministratore/)** □ Pagina di attivazione per gli amministratori, che presenta un modulo in cui l'utente può impostare la propria password per completare l'attivazione dell'account.
- **Amministratori.jsx (Amministratori/)** □ Pagina che consente di visualizzare e gestire gli amministratori, offrendo funzioni per l'eliminazione dell'account e filtri per una selezione mirata.
- **Catalogo.jsx (Catalogo/)** □ Pagina dedicata alla gestione del catalogo dei prodotti, consentendone la visualizzazione e la navigazione al dettaglio. Dispone di funzionalità per il filtraggio e attivazione/disattivazione di prodotti e per la visualizzazione delle statistiche.
- **Categorie.jsx (Categorie/)** □ Pagina per la gestione di categorie e sottocategorie, con funzioni di visualizzazione, aggiunta, modifica, filtro e attivazione/disattivazione.
- **Clienti.jsx (Clienti/)** □ Pagina dedicata alla gestione dei clienti registrati, che consente la visualizzazione dettagliata dei dati anagrafici.
- **DettagliProdotto.jsx (Dettagli Prodotto/)** □ Pagina dedicata alla visualizzazione dettagliata di un prodotto, comprensiva di informazioni generali, dettagli tecnici, media (immagini e video), statistiche di vendita e recensioni dei clienti, con funzionalità di modifica, attivazione/disattivazione e gestione delle recensioni.
- **ModificaProdottoDialog.jsx (Dettagli Prodotto/)** □ Componente che permette l'aggiornamento di tutte le informazioni di un prodotto con visualizzazione di conferma o errore mediante l'uso di un alert.

- **Ordini.jsx (Ordini/)** □ Pagina per la gestione degli ordini, che permette di visualizzare i dettagli cliccando sull'ID e di aggiornare lo stato di ciascun ordine. Include un grafico per le statistiche sugli ordini e mostra chiaramente lo stato dei resi.
- **NivoBarOrdini.jsx (Ordini/)** □ Grafico a barre dei guadagni mensili con selezione dell'anno e adattamento automatico delle etichette dei mesi, gestendo caricamento, errore e assenza di dati.
- **DettaglioOrdine.jsx (DettaglioOrdine/)** □ Pagina che mostra le informazioni complete di un singolo ordine, inclusi articoli, quantità, prezzi, cliente e timeline dello stato. Permette di aggiornare lo stato dei resi e la navigazione al dettaglio dei prodotti.
- **Marche.jsx (Marche/)** □ Pagina per la gestione delle marche, che consente di aggiungere nuovi elementi e filtrare quelli esistenti. Ogni marca presenta dettagli espandibili e comandi rapidi per modificarla o aggiornare il suo stato.
- **NotFoundPage.jsx (NotFoundPage/)** □ Pagina di errore 404 che informa l'utente che la pagina richiesta non esiste o è in fase di sviluppo.
- **Login.jsx (Login/)** □ Pagina di login per l'admin con campi per nome utente e password necessari per l'accesso.
- **Form.jsx (Login/)** □ Componente per la gestione del login, che raccoglie email e password e aggiorna lo stato dell'applicazione con il token di accesso.
- **Ticket.jsx (Tickets/)** □ Pagina per la gestione dei ticket di supporto, che mostra una lista paginata e filtrabile. Permette di visualizzare i dettagli di ciascun ticket cliccando sulla riga e di rispondere direttamente tramite un pop-up dedicato.
- **DettaglioDialog.jsx (Tickets/)** □ Componente di supporto per la visualizzazione dei dettagli di un singolo ticket.
- **RispostaDialog.jsx (Tickets/)** □ Componente per gestire la risposta a un ticket. Mostra l'oggetto e il corpo del ticket selezionato e fornisce un campo di testo per inserire la risposta.
- **Wishlist.jsx (Wishlist/)** □ Pagina per la visualizzazione delle statistiche relative alle wishlist degli utenti.
- **WishlistLineChart.jsx (Wishlist/)** □ Componente utilizzato in Wishlist.jsx per la creazione di un grafico a linea.
- **WishlistMarchePie.jsx (Wishlist/)** □ Componente utilizzato in Wishlist.jsx per la creazione di un grafico a torta sulle marche.

7.1.4 client/src/state/

N.B. Il colore rosso è associato a **Manzoni Matteo** e il colore verde a **Marvulli Antonio** per la divisione delle responsabilità, in caso di responsabilità condivisa il colore sarà neutrale.

Questo percorso contiene tutti i file necessari per la gestione dello stato di Redux e la definizione di endpoint per le chiamate API:

- **api.js** □ File che definisce gli endpoint per le chiamate API al back-end e, in caso di access token scaduto, prova a riottenere il refresh token. In caso di esito negativo reindirizza al logout.
- **globalSlice.jsx** □ File che definisce lo stato globale di redux, gestendo l'autenticazione dell'utente, il tema dell'applicazione, il token di accesso e la query di ricerca.

- **localeText.js** □ File che definisce etichette e testi personalizzati per il DataGrid, traducendo in italiano nomi e messaggi che sarebbero mostrati in inglese.
- **snackbarSlice.jsx** □ File che gestisce lo stato delle notifiche a comparsa (snackbar) nell'interfaccia, permettendo di mostrarle o nasconderle con messaggi e livelli di gravità diversi.

7.2 Server

La cartella server si divide nei seguenti percorsi principali:

- **server/src/config** □ Cartella che contiene diversi file di configurazione dell'applicazione, necessari per utilizzare librerie esterne e stabilire la connessione al database.
- **server/src/controllers** □ Cartella che contiene diversi file con le funzioni che ricevono le richieste dalle API e gestiscono la logica delle operazioni per ogni rotta.
- **server/src/middleware** □ Cartella che contiene funzioni che si interpongono tra la richiesta del client e l'esecuzione del controller per controlli o operazioni preliminari.
- **server/src/models** □ Cartella che contiene diversi file per la definizione dei modelli Mongoose, utilizzati per descrivere la struttura delle collezioni MongoDB, i tipi di dato e le regole di validazione dei documenti.
- **server/src/routes** □ Cartella che contiene diversi file per la definizione delle rotte dell'applicazione, e collegano gli endpoint ai rispettivi controller.
- **server/src/utlis** □ Cartella che comprende funzioni di utilità, impiegate in vari controller.

e dei seguenti file principali:

- **server/src/app.js** □ File principale del server che inizializza l'applicazione Express, configura middleware generici e collega le varie rotte.
- **server/server.js** □ File principale che avvia l'applicazione, si occupa di connettersi al database e di far partire il server Express sulla porta specificata.

8. TEST

Per il test del sistema sono stati utilizzati i seguenti strumenti:

- **Postman** □ Applicazione utilizzata per effettuare dei test funzionali sulle API sviluppate, verificandone il corretto funzionamento simulando richieste HTTP.
- **ESLint** □ Strumento per effettuare l'analisi statica del codice permettendo di individuare e correggere nel codice eventuali bug o errori di sintassi migliorando notevolmente qualità e coerenza del codice.
- **Fake API** □ API utilizzate per simulare informazioni normalmente provenienti dal "lato cliente" dell'applicazione. Le richieste a queste API sono state effettuate tramite Postman per confermare il corretto funzionamento di componenti React front-end.

9. CONCLUSIONI

L'applicazione ha raggiunto uno stato di sviluppo avanzato e implementa un ampio insieme di funzionalità:

- Gestione completa del catalogo, delle marche, delle categorie e sottocategorie
- Consultazione e monitoraggio degli ordini ricevuti sia come lista che come dettaglio
- Possibilità di controllare i ticket di supporto ricevuti con possibilità di fornire una risposta
- Possibilità di controllare i dati di tutti gli utenti registrati al sito web

L'intero sistema è realizzato mediante 42 componenti React e conta circa 17000 righe di codice.

Per completare lo sviluppo è prevista l'implementazione di un chatbot, stimata in ulteriori 2000 righe di codice, che consentirà di migliorare ulteriormente l'esperienza utente.

Ci riteniamo soddisfatti dell'implementazione raggiunta in quanto siamo riusciti a raggiungere quasi tutti gli obiettivi del progetto, lasciando come progetto futuro la realizzazione di un assistente AI per il supporto utente. Tra le tante difficoltà incontrate di sicuro spicca la complessità di integrare tutte le funzionalità del sistema in modo che possano essere manutenibili, scalabili e riutilizzabili.

Il committente è rimasto generalmente soddisfatto del risultato ottenuto con molte aspettative riguardo l'implementazione futura dell'intelligenza artificiale.

In particolare è stato apprezzato l'utilizzo della libreria MUI per ottenere una grafica intuitiva e semplice.

10. MATRICE RACI

Matrice RACI per documentazione del progetto:

<u>ATTIVITÀ</u>	<u>MATTEO MANZONI</u>	<u>ANTONIO MARVULLI</u>
INDIVIDUAZIONE GOAL AMMINISTRATORE (CAPITOLO 2.1)	R	C
DIAGRAMMA DI CONTESTO (CAPITOLO 2.2)	R	C
SCAMBIO DI VALORE (CAPITOLO 2.3)	C	R
STATO DELL'ARTE (CAPITOLO 3.1)	C	R
ANALISI "MAKE OR BUY" (CAPITOLO 3.3)	R	C
DIAGRAMMA EER (CAPITOLO 4.1)	R	C
MODELLO RELAZIONALE (CAPITOLO 4.2)	R	C
STIMA DIMENSIONALE DEL DB (CAPITOLO 4.3)	C	R
MODELLO VISIBILITÀ AMMINISTRATORE (CAPITOLO 4.4)	C	R
MODELLO DI NAVIGAZIONE (CAPITOLO 5.1)	R	C
MODELLO DI PRESENTAZIONE (CAPITOLO 5.2)	R	R
MODELLO ARCHITETTURALE HARDWARE (CAPITOLO 6.1)	C	R
MODELLO ARCHITETTURALE SOFTWARE (CAPITOLO 6.2)	R	A
ASPETTI IMPLEMENTATIVI CLIENT (CAPITOLO 7.1)	A	R
ASPETTI IMPLEMENTATIVI SERVER (CAPITOLO 7.2)	A	R
TEST (CAPITOLO 8)	R	A
CONCLUSIONI (CAPITOLO 9)	C	R

Matrice RACI per sviluppo dell'applicazione web:

<u>ATTIVITÀ</u>	<u>MATTEO MANZONI</u>	<u>ANTONIO MARVULLI</u>
<i>Catalogo</i>	X	
<i>Categorie</i>	X	
<i>Marche</i>	X	
<i>Ordini</i>		X
<i>Analisi wishlist</i>		X
<i>Clienti</i>		X
<i>Ticket</i>		X
<i>Amministratori</i>		X
<i>Login</i>	X	
<i>AI Assistant</i>	TBD	TBD